Université Paris Cité

How Multilingual is Multilingual BERT?

Guillaume Wisniewski
guillaume.wisniewski@u-paris.fr

March 2026

1 Introduction

The goal of this lab, beyond addressing a (to me, at least) genuinely interesting question, is to introduce you to the use of the HuggingFace 🤗 API for fine-tuning a pretrained language model. As discussed during the lecture, this can in principle be achieved in just a few lines of Python.

However, rather than relying on a ready-made dataset from the HuggingFace 🤗 ecosystem, you will construct and transform the data yourself. This is intended to give you a clearer understanding of the complexity underlying certain fundamental steps—most notably tokenisation—which are often overlooked, particularly in multilingual settings.

Some of the technical aspects are not entirely straightforward and will require you to write non-trivial code.

2 Fine-Tuning mBERT

The goal of the first part of this lab is to train (and evaluate) a multilingual PoS tagger by fine-tuning mBERT. To assess the multilingual capacity of mBERT, you will:

- evaluate the performance of a PoS tagger fine-tuned on a given language and compare it to the performance of a PoS tagger trained only on this language;
- evaluate on a language B the performance of a PoS tagger fine-tuned on a language A.

2.1 The Universal Dependencies Project

The Universal Dependencies (UD) project is a collection of treebanks with consistent annotations of grammar (parts of speech, morphological features, and syntactic depen-

dencies) across different human languages. Treebanks are stored in the `conllu` format. Figure 1 shows how to extract tokenized sentences and PoS information from such files.

```
import conllu

def load_conllu(filename):
    for sentence in conllu.parse(open(filename, "rt", encoding="utf-8").read()):
        tokenized_words = [token["form"] for token in sentence]
        gold_tags = [token["upos"] for token in sentence]
        yield tokenized_words, gold_tags

corpus = list(load_conllu("fr_sequoia-ud-test.conllu"))
```

Figure 1: Extracting PoS information from a `conllu` file.

1. Why do we need "consistent annotation" for our experiment?
2. Why do we use the `yield` keyword rather than a simple `return` (line 7)? Why do we have to call the `list` constructor (line 9)?
3. What is the distribution of labels in the test set of the Sequoia treebank? What can you infer from the visualisation of this distribution?
4. What are the multiword tokens in the test set of Sequoia? How are they annotated?
5. Why did the UD project make the decision of splitting multiword tokens into several (grammatical) words? What is your opinion on this decision?
6. According to UD tokenization guidelines, a token can contain spaces. Are there any tokens in the Sequoia corpus that contain spaces? If so, these spaces should be removed.

These questions, which could without hesitation be described as a rather trivial warm-up, are primarily intended to draw your attention to certain specific characteristics of UD corpora. Please make sure to take these into account in all the experiments you carry out.

2.2 mBERT Tokenization

mBERT (as well as the original BERT model) uses a tokenization in subword units. Frequent words are tokenized normally, but infrequent words are split into smaller units which sometimes correspond to affixes. Punctuation is split at the character level. For mBERT, subword tokens that are continuations are prefixed with `##`.

Figure 2 shows how a sentence can be tokenized by the mBERT tokenizer via the HuggingFace API. We will use the `bert-base-multilingual-cased` checkpoint throughout.

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-multilingual-cased")
# show the resulting tokenization
tokenizer.tokenize("What a wonderful world!")
# compute information required by the model
tokenizer("What a wonderful world!")
```

Figure 2: Tokenizing a single sentence with the mBERT tokenizer.

7. Why does mBERT use subword tokenization?
8. How is the sentence *"Pouvez-vous donner les mêmes garanties au sein de l'Union Européene"* tokenized according to the UD convention? How is it tokenized by the mBERT tokenizer?
9. Why is this difference in tokenization a problem for training a mBERT-based PoS tagger?

2.3 Reconciling the Two Tokenizations

To train a PoS tagger, we must provide a PoS label to each token of a sentence tokenized by the mBERT tokenizer. We adopt the following two principles:

- Multiword tokens that must be split according to UD annotation guidelines are kept unchanged and labelled by a new label formed by the concatenation of all their constituent labels. For instance, the French word *au* is not decomposed into *à* (ADP) and *le* (DET) but labelled `ADP+DET`.
- For UD tokens that are sub-tokenized into several units by mBERT, the tag is aligned with the first subtoken of the word; all remaining subtokens receive the special label `<pad>`. This label will also be used for padding tokens and will be ignored during evaluation.

Table 1 illustrates the result of applying these two principles.

To implement the second principle, set `return_offsets_mapping=True` when calling the tokenizer. For each subtoken, the offset mapping returns a tuple `(start, end)` relative to the original token: if `start` $\neq$ `0`, the subtoken is a continuation and its label

Level	Tokens / Labels
Raw sentence	`Revenons-en aux choses essentielles.`
UD tokenization	`Revenons -en à les choses essentielles .`
UD original labels	`VERB PRON ADP DET NOUN ADJ PUNCT`
UD updated tokenization	`Revenons -en aux choses essentielles .`
UD updated labels	`VERB PRON ADP+DET NOUN ADJ PUNCT`
mBERT tokenization	`Rev ##eno ##ns - en aux choses essentiel ##les .`
Labels for mBERT	`VERB <pad> <pad> PRON <pad> ADP+DET NOUN ADJ <pad> PUNCT`

Table 1: Example of tokenization alignment.

must be `<pad>`; if both `start` and `end` are 0, the token is a special token (`[CLS]`, `[SEP]`, etc.) and its label must also be `<pad>`.

Sentences must also be padded to enable batching. The tokenizer should be called as follows:

```
tokenizer(texts, is_split_into_words=True,
          return_offsets_mapping=True,
          padding=True,
          truncation=True)
```

where `texts` is a list of lists of strings (one list per sentence).

In HuggingFace 🤗, labels must be represented as integers; the special `<pad>` label must be mapped to -100. This value is automatically ignored by PyTorch loss functions, *and must also be explicitly excluded when computing accuracy*: make sure your evaluation metric only considers positions where the gold label is not -100.

Note also that the `truncation=True` option silently discards tokens beyond the model's maximum sequence length (512 subword tokens for mBERT). For long sentences, the labels corresponding to truncated tokens are simply dropped.

10. Write a function that takes as input a sentence tokenized according to UD rules (a `List[str]`) and its gold labels (also a `List[str]`) and implements the first principle described above.

11. Write a function that takes as input the sentences and labels produced by normalizing UD sentences, applies the mBERT tokenizer, and computes the corresponding label sequence. Every subtoken (including padding symbols) must have a label.

12. What is the practical effect of `truncation=True` on the labels? How frequent is truncation in the corpora you are using? Could it bias the evaluation, and if so, how?

13. Write a function that encodes the labels into integers.

2.4 Creating a Dataset

Now that we have a labeled dataset adapted to the mBERT tokenization, we convert it into a format suitable for training. We encapsulate the data in a HuggingFace `Dataset` object, which can be seen as a list of dictionaries. The keys used must match the argument names expected by the model. For mBERT, the required keys are: `input_ids`, `attention_mask`, and `labels`.

14. Using `Dataset.from_list`, write a method that creates a `Dataset` encapsulating a corpus in the `conllu` format.

15. Create three `Dataset` instances: one for the train set, one for the dev set, and one for the test set.

2.5 Fine-Tuning mBERT

Figure 3 shows how to fine-tune a model using the HuggingFace `Trainer` API. All hyperparameters are encapsulated in a `TrainingArguments` instance; once these are set, fine-tuning reduces to a single method call.

16. How can you modify the code of Figure 3 to report PoS tagging accuracy during training? Why is this information important?

3 Evaluating the Multilingual Capacity of mBERT

Choosing your languages. You are free to choose any 5 languages from the UD project, but your choice must be linguistically motivated. Before deciding, read the original paper by [Pires et al.(2019)] and look critically at the languages they selected.

17. What languages did the original paper use to evaluate the multilingual capacity of mBERT? What is wrong with this choice from a linguistic point of view?

Corpus size. UD treebanks vary enormously in size. Before training, report the number of sentences and tokens in the train, dev, and test splits of each language you have selected.

18. Compare the sizes of your training corpora. Is it reasonable to compare the results obtained on languages with very different amounts of training data? How could corpus size confound the conclusions you draw about multilinguality?

Experiments.

19. For each of your 5 languages, train a PoS tagger using the code from the previous section and evaluate it on the test sets of all 5 languages. Report your results in a 5×5 matrix (rows = training language, columns = test language).

20. Can you use the same hyperparameters for all languages? What should you do to make the comparison fair?

21. Analyse your result matrix. Which transfer directions work well? Which fail? Can you relate the pattern of results to the linguistic properties of the languages involved (family, morphology, script)?

References

[Pires et al.(2019)] Telmo Pires, Eva Schlinger, and Dan Garrette. How multilingual is multilingual BERT? In *Proceedings of ACL*, 2019.

```
batch_size = 16

from transformers import (AutoModelForTokenClassification,
                          TrainingArguments, Trainer)

model_checkpoint = "bert-base-multilingual-cased"
model = AutoModelForTokenClassification.from_pretrained(
    model_checkpoint, num_labels=len(label_list))

training_args = TrainingArguments(
    output_dir="to_be_chosen",
    evaluation_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=batch_size,
    per_device_eval_batch_size=batch_size,
    num_train_epochs=3,
    weight_decay=0.01,
    logging_dir='./logs',
    logging_steps=10,
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=ds,    # train set
    eval_dataset=ds      # dev set
)

trainer.train()
```

Figure 3: Fine-tuning a model with the HuggingFace `Trainer` API.